

Face Recognition Attendance System

MediaPipe + FaceNet Powered Automatic Attendance Platform

INTERVIEW PREPARATION GUIDE

A complete walkthrough of the project — what it does, how it works internally, the database design, authentication, the full tech stack, architecture, scaling roadmap, real-world use cases, and a recruiter-style Q&A bank.

Personal / Academic Project

Prepared August 2026

1 The Elevator Pitch (memorize this, then let the interviewer steer)

Keep this to about 90 seconds. Say it in three beats — the problem, what you built, how it works — then pause and let the interviewer ask a follow-up.

“This is a web-based attendance system that recognizes students automatically from a webcam feed instead of relying on roll call or manual marking. A student looks at the camera, and if their face matches someone registered in the system, their attendance is marked instantly — no card, no fingerprint, no manual entry. Under the hood, it's a two-model pipeline: I use Google's MediaPipe BlazeFace model to detect and crop the face out of each webcam frame, then I feed that crop into FaceNet, which converts the face into a 512-dimensional numerical embedding — basically a fingerprint of that person's facial features. During registration, I capture 15 to 20 frames, generate an embedding for each one, and average them into one representative embedding per student, stored in PostgreSQL. During recognition, I do the same thing for a single live frame and compare that embedding against every stored embedding using cosine similarity — if the closest match is above a similarity threshold, attendance gets marked automatically. The nice part about this embedding-based approach, compared to the older LBPH classifier method it replaced, is that there's zero retraining — you can register a brand-new student and recognize them immediately, because I'm not training a single shared classifier model, I'm just comparing vectors. The whole thing is a Flask backend with PostgreSQL for storage, Flask-Login for admin authentication, and a plain JavaScript/WebRTC frontend that captures webcam frames and posts them to the backend as base64 images.”

Why this project is a strong interview story

- It's a real **applied computer vision + machine learning** system — face detection, embeddings, similarity search — not just a CRUD app with a login page bolted on.
- It shows you understand **why** an architecture choice matters: the whole project is built around the insight that embedding-based recognition removes the retraining bottleneck of older classifier-based approaches — a genuinely good engineering story to tell.
- It has a **proper relational database design** (four related tables, foreign keys, cascading deletes) — good material for database and backend questions.
- It has **real, honest limitations** you can discuss (a brute-force search over all embeddings, a hardcoded admin bootstrap, no liveness detection) — naming these yourself is a strong signal.
- It's **demoable** — you can literally register your own face and mark your own attendance live in an interview if you have a laptop with a webcam.

2 What the Project Actually Does

It's a web dashboard for schools/colleges/offices that replaces manual roll-call attendance with automatic, face-recognition-based attendance. An admin logs in, registers students (including capturing their face data), and then attendance gets marked automatically whenever a registered student's face is recognized by the webcam — no manual check-off needed.

Capability	What it does
Student management	Add, edit, delete, and search student records (name, roll number, department, course, contact info, etc.).
Face registration	Capture 15–20 webcam frames of a student's face, convert them into one stored 'face fingerprint' (embedding) — no photos saved, no model retraining.
Automatic attendance marking	Show your face to the webcam; if it matches a registered student above a similarity threshold, attendance is marked instantly, once per day.
Attendance records & reporting	View, filter (by date/department), manually correct, export to CSV, and import attendance from CSV.
Admin authentication	Secure login/registration for staff/admin accounts, with a password-reset flow.
Dashboard	At-a-glance stats — total students registered, today's attendance count.

3 Why This Is Better Than the Obvious Alternatives

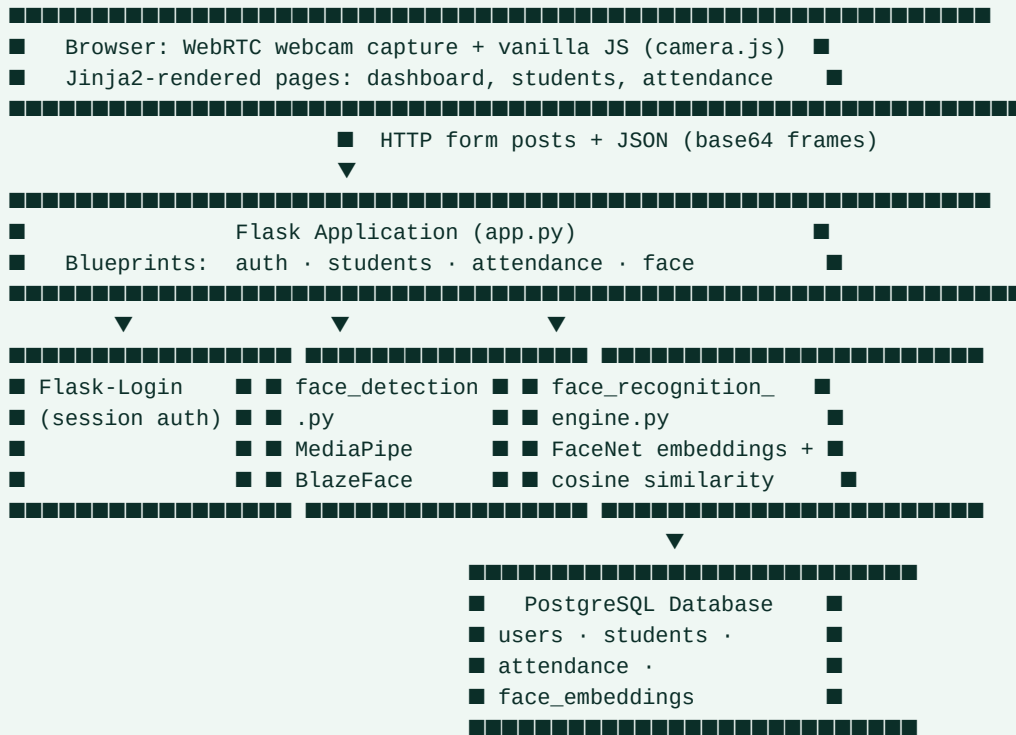
Alternative	Why it falls short
Manual roll call / signature sheets	Slow (minutes per class), easy to fake ('proxy attendance' — one student signing for an absent friend), and error-prone to tally by hand.
RFID cards / barcode scanning	Requires everyone to physically carry and tap a card — cards get lost, forgotten, or lent to someone else, which defeats the whole point of verifying identity.
Fingerprint biometric scanners	Requires dedicated hardware at every entry point, is a contact-based (hygiene-sensitive) interaction, and scales poorly to large classes since students must queue one at a time.
Legacy face recognition using OpenCV Haar cascades + an LBPH classifier	This is literally what an earlier version of this project used, and it's genuinely worse: Haar cascades are noticeably less accurate under bad lighting or off-angle faces, and LBPH is a single trained classifier — every time you add a new student, you have to retrain the whole model from scratch before it can recognize them.

The core idea that makes the current design better: using deep-learning face embeddings (FaceNet) instead of a trained classifier means recognition is just *vector comparison* — adding a new student is instant, because you're not retraining anything, you're just adding one more fingerprint to compare against.

4 High-Level Architecture

It's a classic server-rendered web application: a browser captures webcam frames with WebRTC/JavaScript and sends them to a Flask backend, which runs the two-stage face pipeline (detect, then embed), compares against PostgreSQL, and returns a result — all rendered through Jinja2 templates plus a few JSON API endpoints for the live camera interactions.

4.1 Block diagram



4.2 The two flows that matter most

Flow	Trigger	What happens
Registration	POST /face/capture-photos/	15–20 webcam frames → detect face in each → generate FaceNet embedding for each → average into one vector → store in face_embeddings table
Recognition / attendance	POST /face/recognize	1 webcam frame → detect face(s) → generate embedding(s) → compare against every stored embedding (cosine similarity) → mark attendance for the best match above threshold

5 What's Happening Internally, Step by Step

5.1 Registering a new student's face

- 1 An admin fills in the student's details (name, roll number, department, etc.) through the students blueprint, which creates a row in the students table with photo_sample = 'No'.
- 2 The admin then opens the face-capture screen, which uses WebRTC to access the webcam and JavaScript to grab 15–20 frames over a few seconds, encoding each as a base64 image string.

- 3 Those frames are POSTed as JSON to `/face/capture-photos/<student_id>`. For each frame: it's decoded from base64 back into an image, run through `FaceDetector.crop_faces()` (the MediaPipe wrapper), which returns the cropped face region if one is found — frames with no detectable face are simply skipped and logged.
- 4 Each successfully cropped face is passed to `generate_embedding()`, which resizes it to FaceNet's expected 160×160 input, runs it through the FaceNet model to get a 512-number vector, and L2-normalizes it (scales it so its length is exactly 1) — this normalization is what lets cosine similarity later be computed as a simple dot product instead of a more expensive calculation.
- 5 Once every frame has been processed, all the valid embeddings collected are averaged together into a single representative 512-D vector via `average_embeddings()` — this smooths out noise from any one bad frame (blurry, weird angle, blinking) by combining signal across many frames.
- 6 Any previous embedding for that student is deleted (so re-registering a student cleanly replaces their old face data), the new averaged embedding is serialized to a JSON string and saved as a new row in the `face_embeddings` table, and the student's `photo_sample` flag is flipped to 'Yes'.
- 7 Notice what's *not* happening: no photo files are saved to disk, and there's no model training step at all — this is the key architectural difference from the old LBPH-based version, where adding a student meant saving grayscale JPEGs and retraining a shared classifier file.

5.2 Recognizing a face and marking attendance

- 1 The live-attendance screen continuously captures single webcam frames and POSTs each one (base64 encoded) to `/face/recognize`.
- 2 The backend decodes the frame, runs MediaPipe face detection/cropping again, and if no face is found, immediately returns a friendly 'no face detected' response — no wasted embedding computation.
- 3 If a face is found, it generates a 512-D embedding for it (the exact same function used at registration time — this consistency is essential for the comparison to be meaningful).
- 4 Every stored embedding is loaded from the `face_embeddings` table (one row per registered student), and `find_best_match()` compares the live embedding against every one of them using cosine similarity, keeping track of whichever student scores highest.
- 5 If the best match's similarity score clears the configured threshold (0.6 by default — configurable via environment variable), it's accepted as a match; otherwise no match is returned, even if it was the 'closest' one available, because a low score means it probably isn't actually that person.
- 6 For a matched student, the system checks whether they've already been marked present today; if not, a new Attendance row is created with the current date/time and status 'Present'. If they were already marked, the response simply says so — this prevents duplicate attendance rows from someone standing in front of the camera for several seconds.
- 7 The response includes the matched student's name, roll number, department, and a human-readable confidence percentage, so the frontend can show a clear 'Attendance marked for (92% confidence)' message.

6 Database Deep Dive — What, Why, and How the Tables Connect

6.1 Why PostgreSQL

- **Relational integrity:** attendance and face-embedding rows are always tied to a real, existing student through foreign keys — the database itself enforces this, instead of relying on application code to never make a mistake.
- **ACID transactions:** attendance records are the whole point of the system, so guaranteeing that a 'mark attendance' operation either fully succeeds or fully rolls back (never leaves a half-written row) matters a lot for data you might use for grading or payroll.
- **Cascading deletes:** when a student is removed, PostgreSQL automatically removes their face embeddings and attendance history too (via `ON DELETE CASCADE`), instead of leaving orphaned rows behind.
- **Production-grade and free:** PostgreSQL handles concurrent reads/writes well (multiple admins using the dashboard at once), and integrates cleanly with Flask through Flask-SQLAlchemy and the psycopg2 driver.
- **Good enough for storing embeddings as-is:** a 512-number vector is just serialized to a JSON string and stored in a plain `TEXT` column — no exotic data type needed at this scale (though this is also a scaling limitation worth naming — see §12).

6.2 The four tables

Table	Purpose	Key columns
users	Admin/staff accounts for logging into the dashboard.	id (PK), username (unique), email (unique), password_hash, security_question, security_answer, created_at
students	One row per enrolled student — their profile info.	id (PK), student_id (unique business key), name, roll_no, department, course, year, semester, gender, contact info, photo_sample (Yes/No), created_at
attendance	One row per attendance event (a student present on a given date).	id (PK), student_id (FK → students.student_id), roll_no, name, department (denormalized for fast CSV export), time, date (indexed), status, created_at
face_embeddings	One row per student's stored face 'fingerprint'.	id (PK), student_id (FK → students.student_id, ON DELETE CASCADE), embedding (TEXT — JSON array of 512 floats), created_at

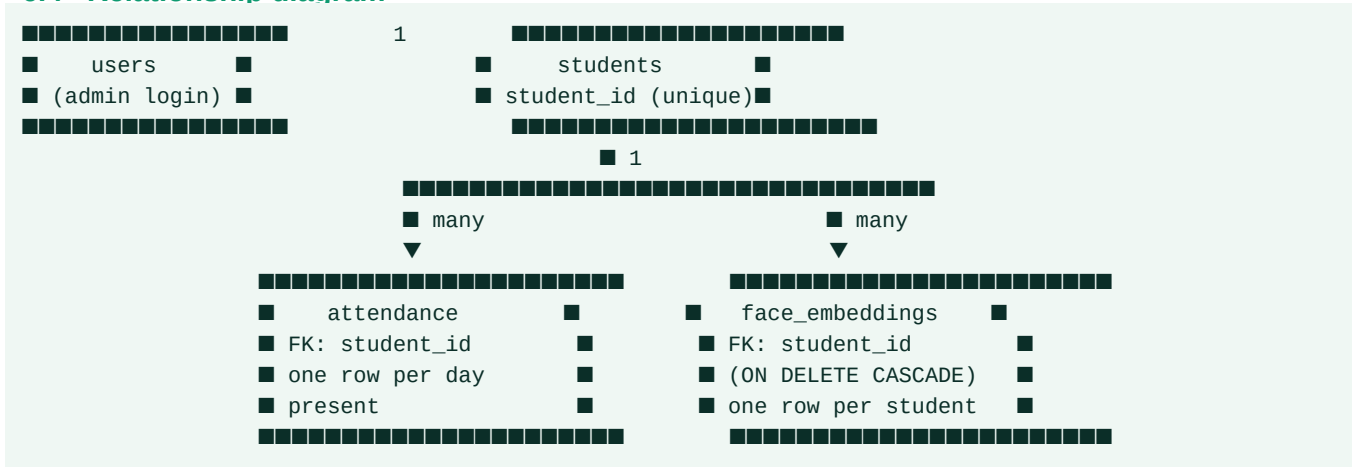
6.3 How the tables connect

The `students` table is the hub. Both `attendance` and `face_embeddings` reference it through a foreign key on `student_id` — note that this is the human-readable **business key** (a string like `'STU2024001'`), not the internal numeric `id` primary key. This is a deliberate (if slightly unconventional) design choice: it makes CSV export/import and manual queries more readable, at a small cost in join performance compared to using the integer primary key.

Two relationships are defined from `Student`: `attendance_records` and `face_embeddings`, both set with `cascade='all, delete-orphan'` at the SQLAlchemy (application/ORM) level, *and* the `face_embeddings` foreign key additionally has `onDelete='CASCADE'` enforced directly at the database level. In practice this means

deleting a student cleanly removes all their attendance history and face data too, whether that delete happens through the ORM or directly in the database — a nice belt-and-suspenders design worth mentioning if asked about data integrity.

6.4 Relationship diagram



7 Login & Authentication — How It Actually Works

- 1 **Flask-Login** handles the session side of authentication. The `User` model inherits from `UserMixin` (which gives it the properties Flask-Login expects, like `is_authenticated`), and a `@login_manager.user_loader` function tells Flask-Login how to reload a `User` object from the ID stored in the session cookie on every request.
- 2 **Passwords are never stored in plain text.** Werkzeug's `generate_password_hash()` hashes the password before it's saved to the `password_hash` column, and `check_password_hash()` verifies a login attempt against that hash — the raw password itself is never persisted anywhere.
- 3 **Logging in** (`POST /login`) looks up the user by username, verifies the password with `check_password_hash()`, and calls Flask-Login's `login_user()`, which sets a signed session cookie (signed using the app's `SECRET_KEY`) marking that browser as logged in. The login route also supports 'remember me' (a longer-lived persistent cookie) and redirects back to whatever page the user originally tried to reach before being sent to the login screen.
- 4 **Protecting pages** is done with the `@login_required` decorator on every route that shouldn't be reachable by a logged-out visitor (all of the students, attendance, and face routes use it) — Flask-Login automatically redirects anyone not logged in to the login page.
- 5 **Registration** (`POST /register`) validates that all fields are filled in, passwords match, the terms checkbox is ticked, and the email/username aren't already taken, before hashing the password and creating a new `User` row.
- 6 **Password reset** uses a security-question/security-answer flow instead of an email reset link: the user provides their email and answer, and if it matches what's on file, they can set a new password directly.
- 7 **A first-run convenience worth knowing about:** the login route has a special case — if someone logs in with the literal username/password `admin / admin`, it automatically creates an admin account in the database if one doesn't exist yet, so the very first login always works out of the box without a separate seeding step. This is genuinely convenient for a class project or demo, but it's a real security issue if the app were ever deployed publicly without removing or disabling it — a good limitation to name yourself (see §12).

8 Tech Stack — What, and Why Each Piece Was Chosen

Technology	Role / why it was used
Python + Flask	The web framework running the whole backend. Flask was chosen for being lightweight and explicit — the app factory pattern (<code>create_app()</code>) plus Blueprints keep auth, students, attendance, and face-recognition code cleanly separated into their own modules.
Flask-SQLAlchemy	The ORM (Object-Relational Mapper) — lets database tables be defined and queried as normal Python classes/objects instead of writing raw SQL everywhere.
PostgreSQL + psycopg2	The production-grade relational database storing users, students, attendance, and face embeddings — see §6 for the full reasoning.
Flask-Login	Handles admin session authentication — who's logged in, protecting routes, remember-me cookies. See §7.
Werkzeug security helpers	Provides the password hashing (<code>generate_password_hash</code> / <code>check_password_hash</code>) used for admin accounts — passwords are never stored in plain text.
Google MediaPipe Tasks (BlazeFace)	The face <i>detection</i> model — finds and draws a bounding box around any face in a frame. Runs as a small on-device .tflite model (<code>blaze_face_short_range.tflite</code>), fast enough for real-time webcam use, and notably more robust to lighting/angle than the older Haar cascade approach it replaced.

Technology	Role / why it was used
Keras-FaceNet (TensorFlow backend)	The face <i>recognition</i> model — takes a cropped face and outputs a 512-dimensional embedding representing that person's facial identity, used for comparison instead of a retrained classifier.
NumPy	Used throughout the recognition engine for vector math — normalizing embeddings, computing cosine similarity (a dot product) and Euclidean distance.
python-dotenv	Loads configuration (database credentials, secret key, thresholds) from a local .env file instead of hardcoding secrets into the source code.
WebRTC + vanilla JavaScript (camera.js)	Runs entirely in the browser to access the webcam and capture frames — no video is ever streamed continuously to the server; individual frames are captured and sent as needed.
Jinja2 templates	Server-rendered HTML pages (dashboard, student list, attendance log, forms) — a classic server-rendered app rather than a separate single-page frontend framework.

Interview-ready one-liner on the stack choice: “I split face *detection* and face *recognition* into two separate, purpose-built models — MediaPipe for finding a face fast, FaceNet for identifying whose face it is — instead of using one general-purpose model to do both, because each model is genuinely better at its one specific job.”

9 How the Backend Actually Runs

It's a single Flask process using the **application factory pattern**: `create_app(config_name)` builds and configures the app, rather than creating a global app object at import time — this makes it easy to spin up different configured instances (e.g. for testing) later.

Startup sequence

- 1 `create_app()` loads the right config class (Development or Production) from `config.py`, which reads database credentials and settings from environment variables (via `python-dotenv`).
- 2 Extensions are initialized: `db.init_app(app)` (SQLAlchemy) and `login_manager.init_app(app)` (Flask-Login).
- 3 Required folders are created if missing (upload folder, data folder, model folder, embeddings folder).
- 4 The four Blueprints — `auth`, `students`, `attendance`, `face` — are registered, each mounted under its own URL prefix (e.g. everything under `/students/...` is handled by the `students` blueprint).
- 5 `db.create_all()` runs inside an app context, creating any database tables that don't already exist — so the very first run of the app automatically sets up the schema, no manual migration step needed for a fresh install.
- 6 The dev server starts on `0.0.0.0:5001` via `app.run(debug=True)` when run directly with `python app.py`.

Where the ML models live

Both ML models are loaded as **module-level singletons** — `get_detector()` and `get_facenet()` only actually construct the (relatively heavy) MediaPipe and FaceNet models the first time they're called, and every subsequent call reuses the same in-memory instance. This matters a lot for performance: loading a deep learning model from disk takes real time, so doing it once per server process instead of once per request keeps recognition fast.

Request-response model

Unlike the WaveSense-style real-time dashboards you may be familiar with, this project doesn't use WebSockets or Server-Sent Events — the camera page simply POSTs one frame at a time to `/face/recognize` (typically on a short interval or button press) and gets a plain JSON response back. It's a simple, synchronous request/response pattern — easy to reason about, though it does mean the browser is responsible for deciding how often to sample the webcam.

10 Advantages

- **Zero retraining for new students** — because recognition is vector comparison, not a trained classifier, a newly registered student can be recognized immediately.
- **No stored photos** — only mathematical embeddings are saved, not actual face images, which is friendlier from a privacy standpoint than keeping a folder of everyone's photos on disk.
- **Much more robust detection** than the legacy Haar-cascade approach — MediaPipe's BlazeFace handles varied lighting and angles noticeably better.
- **Clean separation of concerns** — detection (MediaPipe) and recognition (FaceNet) are two independent, swappable modules, and the whole app is organized into clear Flask Blueprints by feature.
- **Real relational integrity** — cascading deletes and foreign keys mean the database can't easily end up in an inconsistent state.
- **Prevents duplicate attendance** — the 'already marked today' check stops the same student being logged multiple times from repeated camera frames.

- **Configurable similarity threshold** — the strictness of matching can be tuned via an environment variable without touching code.
- **CSV import/export** — attendance data can be pulled into spreadsheets for reporting, or bulk-loaded from an existing system.

11 Real-World Applications

Use case	How this project applies
School / college daily attendance	The core use case — automatically mark attendance as students enter a classroom, eliminating roll call and proxy attendance.
Corporate office check-in	Replace badge-swipe or sign-in-sheet systems for employee time-and-attendance tracking.
Exam hall identity verification	Confirm the person sitting for an exam matches their registered identity before allowing entry, reducing impersonation.
Event / conference check-in	Fast, contactless check-in for pre-registered attendees at a large event.
Gym / fitness club membership check-in	Automatic member verification at entry without needing a physical membership card.
Secure facility or lab access logging	Combine identity verification with an access log, showing exactly who entered a restricted area and when.
Hybrid/online exam proctoring	A lightweight identity check at the start of an online exam session, verifying the test-taker matches their registered face.

12 Limitations (know these — a recruiter will ask, and naming them yourself is a strength)

Limitation	Why it matters
Brute-force search over every stored embedding	recognize() loads and compares against every single student's embedding on every request — fine for a class of 50, but this is $O(n)$ and will slow down noticeably once you get into the thousands of students.
One averaged embedding per student	A single vector can't fully capture how someone looks across different lighting, angles, glasses on/off, or years of aging — recognition quality can drift over time without periodic re-registration.
No liveness/anti-spoofing detection	The system checks 'does this face match,' not 'is this a real, live person' — in principle a printed photo or a phone screen held up to the camera could be matched if it's a good enough image of a registered student.
Hardcoded admin/admin bootstrap in the login route	Convenient for first-run demos, but a genuine security hole if ever deployed publicly without removing it — anyone can log in as admin on a fresh install.
Security question/answer stored and compared in plain text	A weaker password-reset pattern than it should be — the security answer isn't hashed the way the login password is.
Debug mode enabled by default (app.run(debug=True))	Flask's interactive debugger can allow arbitrary code execution if it's ever exposed on a public network — fine for local development, dangerous in production.
No rate limiting on login or recognition endpoints	Nothing currently stops repeated automated login attempts, or hammering the recognition endpoint with frames.
Foreign keys use the string student_id, not the integer primary key	Slightly less efficient for database joins/indexes at scale compared to referencing the numeric id directly — a deliberate readability trade-off worth being able to justify.

How to frame these in an interview: pick one or two and say what you'd do about them — e.g. “The brute-force embedding search is fine at class scale, but I'd move to an approximate nearest-neighbour index like pgvector or FAISS once the student count grew into the thousands, so recognition stays fast without scanning every row.”

13 Improvements & How to Scale This to Industry Level

13.1 Fixes and improvements to the current design

Area	Improvement
Faster similarity search at scale	Move from a brute-force Python loop to pgvector (a PostgreSQL extension purpose-built for storing and indexing vectors) or a dedicated vector database (FAISS/Milvus), so matching stays fast even with tens of thousands of registered faces.
Multiple embeddings per student	Store several embeddings per student (not just one average) captured under different conditions, and match against the best of them — improves robustness to lighting/angle/appearance changes.
Liveness / anti-spoofing	Add a lightweight liveness check (e.g. requiring a blink, or a simple texture/frequency-based photo-attack detector) before accepting a match, so a printed photo can't be used to mark someone else present.
Remove the hardcoded admin bootstrap	Replace the 'admin/admin auto-creates an account' shortcut with a proper one-time seed script or setup wizard, so there's never a default credential sitting in the codebase.
Hash the security answer	Store the password-reset security answer the same way passwords are stored — hashed, not plain text — or move to an email-based reset link instead.
Rate limiting	Add request throttling (e.g. Flask-Limiter) on the login and face-recognition endpoints to prevent brute-force login attempts and endpoint abuse.
Background/async processing for recognition	Move face detection + embedding generation off the main request thread (e.g. into a task queue like Celery/RQ, or an async worker) so one slow inference doesn't block other requests when multiple cameras are active at once.
Pagination on large tables	Add pagination to the student list and attendance log so performance stays good once there are thousands of records instead of loading every row on every page view.
Move ML inference into its own service	Split the MediaPipe/FaceNet inference into a separate internal API so the web app tier and the (heavier, possibly GPU-backed) inference tier can scale independently.

13.2 Scaling to an industry-grade product

- 1 **Multi-classroom / multi-campus support:** tag students, embeddings, and attendance records with a school/branch/classroom ID, and support many cameras streaming concurrently through a queue-based ingestion pipeline instead of one browser tab posting one frame at a time.
- 2 **Role-based access control:** move beyond a single 'admin' concept to proper roles (teacher, department head, super-admin) with different permissions, and per-action audit logging for accountability.
- 3 **Caching hot data:** cache today's attendance stats (used on the dashboard) in something like Redis instead of re-querying PostgreSQL on every page load.
- 4 **Containerized, horizontally scalable deployment:** package the app with Docker, run multiple app instances behind a load balancer, and move session storage to a shared store (e.g. Redis-backed sessions) so login state works correctly across instances.
- 5 **Mobile companion app:** expose the recognition/attendance endpoints behind token-based auth (JWT) in addition to the existing cookie-based Flask-Login session, so a native mobile check-in app could reuse the same backend.
- 6 **Continuous model quality monitoring:** track recognition confidence trends per student over time and flag anyone whose match confidence is steadily dropping — a good proxy for 'this person's stored face data is stale and needs re-registration.'

- 7 **Proper CI/CD and automated tests:** add test coverage for the recognition pipeline (using saved sample embeddings) and the attendance logic (duplicate-marking prevention, CSV import/export correctness) before deploying changes.

14 Interview Question Bank (Recruiter-Style, With Answers)

14.1 Project Overview / Behavioral

Q: Tell me about this project in your own words.

A: Use the pitch in §1 — the problem, what you built, then a quick technical walkthrough of the two-model pipeline. Then pause and let the interviewer steer.

Q: What was the hardest part of building this?

A: Getting reliable face detection and embedding quality from a live, uncontrolled webcam feed — lighting, angle, and motion blur all affect a single frame's usefulness. That's exactly why I capture 15-20 frames at registration and average their embeddings instead of trusting just one photo — it smooths out the noise from any individual bad frame.

Q: What would you do differently if you started over?

A: I'd design the embedding storage and lookup around a proper vector index (like pgvector) from day one, instead of a brute-force loop over a JSON column — it works fine at small scale, but it's the first thing that would need to change for a larger deployment.

Q: How is this better than the older LBPH/Haar cascade version it replaced?

A: Two separate improvements: MediaPipe's BlazeFace model detects faces more reliably under varied lighting and angles than Haar cascades did, and FaceNet-based embeddings mean recognizing a new student needs no retraining at all — with LBPH, adding one new student meant retraining a shared classifier model on the entire dataset again before it could recognize anyone new.

14.2 Computer Vision / ML Questions

Q: Walk me through what happens when a student registers their face.

A: The browser captures 15-20 webcam frames over WebRTC and sends them as base64 images. For each one, MediaPipe detects and crops the face; FaceNet then converts that crop into a 512-dimensional, L2-normalized embedding. All the valid embeddings get averaged into one representative vector, which is saved to the `face_embeddings` table — no images are stored, just that final numeric vector.

Q: Why use two separate models (MediaPipe and FaceNet) instead of one model that does both detection and recognition?

A: Each model is purpose-built and genuinely better at its one job — MediaPipe's BlazeFace is optimized specifically for fast, accurate face *localization*, while FaceNet is trained specifically to produce embeddings that place the same person's face close together in vector space regardless of the photo. Splitting detection and recognition into two specialized stages is actually the standard approach in production face-recognition systems.

Q: What is a face embedding, in your own words?

A: It's a fixed-length list of numbers — 512 of them here — that represents a face's distinguishing features in a way that's comparable mathematically. Two photos of the same person produce embeddings that are close together (high cosine similarity); two different people produce embeddings that are far apart. It converts 'is this the same person' into a math problem.

Q: Why L2-normalize the embeddings?

A: Once a vector is L2-normalized (scaled to have a length of exactly 1), cosine similarity between two normalized vectors becomes just their dot product — a much cheaper calculation than the full cosine similarity formula, which matters when you're comparing against every stored student on every recognition request.

Q: Why average multiple embeddings instead of just using one photo?

A: A single frame can be a bad representative of someone's face — motion blur, an odd angle, an awkward expression, uneven lighting. Averaging embeddings from 15-20 frames produces a more stable, representative 'center point' for that person's face, which is more robust for matching against future live frames taken under different conditions.

Q: How do you decide whether a match is 'good enough'?

A: A configurable similarity threshold — 0.6 by default. The best-matching student's cosine similarity score has to clear that threshold to be accepted; otherwise, even the closest match found is rejected, because a low score means it's probably not actually a match, just the least-different option among people who genuinely aren't that person.

Q: What would happen if two students look very similar (e.g. identical twins)?

A: Their embeddings would likely be close together, which increases the risk of a false match, especially near the similarity threshold. In a real deployment I'd want a lower default threshold plus operator awareness for cases like that, and possibly a secondary verification step (a manual override or a second factor) for flagged edge cases.

14.3 Database Questions

Q: Why did you choose PostgreSQL specifically?

A: It's a mature, production-grade relational database with strong support for foreign keys, cascading deletes, and transactions — all of which matter here, since attendance and face-embedding rows always need to stay correctly tied to a real student record. It also integrates cleanly with Flask through Flask-SQLAlchemy.

Q: Walk me through your database schema and how the tables relate.

A: Four tables: users for admin login, students as the central table holding profile data, attendance with one row per present-day record, and face_embeddings with one stored vector per student. Both attendance and face_embeddings reference students through a foreign key on the student_id business key, and deleting a student cascades to remove their attendance history and face data automatically.

Q: Why use the string student_id as the foreign key instead of the integer primary key?

A: It was a readability trade-off — using the human-readable student ID makes CSV export/import and direct database inspection much easier to follow than chasing numeric IDs. The cost is a small amount of extra join/index overhead compared to using the integer primary key directly, which would be the more conventional choice at larger scale.

Q: How are face embeddings actually stored in the database?

A: As a JSON-serialized string of 512 floating-point numbers, in a plain TEXT column. It's simple and works well at small scale, but it means finding the best match requires loading and comparing every row in Python rather than letting the database do indexed similarity search — a real limitation I'd address with a proper vector extension like pgvector as the system grew.

Q: What happens at the database level when you delete a student?

A: Both the ORM relationships (defined with cascade='all, delete-orphan' in SQLAlchemy) and the database foreign key itself (defined with ON DELETE CASCADE) ensure that deleting a student automatically removes their attendance records and face embedding — enforced doubly, both at the application layer and the database layer.

14.4 Backend / Auth Questions

Q: How does login actually work here?

A: Flask-Login manages the session: on successful login (password checked against a hashed value using Werkzeug's check_password_hash), a signed session cookie is set identifying that user. Every protected route uses the @login_required decorator, which checks that cookie and redirects to the login page if it's missing or invalid.

Q: Are passwords stored securely?

A: Yes for the login password — it's hashed with Werkzeug's generate_password_hash before being saved, and the raw password is never stored. The password-reset security answer, however, is currently stored and compared in plain text, which is a real weak point I'd fix by hashing it too, or switching to an email-based reset link entirely.

Q: I noticed there's a hardcoded admin/admin login shortcut — is that a security risk?

A: Yes, and it's worth naming directly rather than hoping it doesn't come up: it's a convenience for first-run demos, since it auto-creates an admin account on the very first login attempt, but it's a genuine security hole if the app were ever deployed publicly without removing it — anyone could log in as admin on a fresh install. I'd replace it with a proper one-time setup script before any real deployment.

Q: Why Flask over Django here?

A: Flask's lightweight, unopinionated structure fit this project well — it's a relatively focused app (auth, students, attendance, face recognition), and Flask's Blueprint system was enough to keep those concerns cleanly separated without needing Django's larger built-in ecosystem (admin panel, full ORM migrations framework, etc.) that this project doesn't otherwise need.

14.5 Scaling / Critical Thinking Questions

Q: What's the single biggest weakness of this project as it stands today?

A: The brute-force embedding comparison — `recognize()` scans every stored student's embedding on every request. It's completely fine for a classroom of dozens of students, but it doesn't scale cleanly to thousands, which is exactly the kind of thing I'd fix first by moving to an indexed vector search.

Q: How would you scale this to a whole school district with tens of thousands of students?

A: Move similarity search to an indexed vector store (pgvector or a dedicated vector database) instead of a linear scan, add caching for dashboard stats, move ML inference into its own horizontally-scalable service separate from the web app, and add proper multi-campus/classroom scoping so searches only compare against the relevant subset of students rather than the entire district at once.

Q: If you had a week to improve this project, what would you do?

A: Two things with the best effort-to-value ratio: remove the hardcoded admin/admin login shortcut and replace it with a proper setup step (a real security fix), and add a basic liveness check so a printed photo can't be used to spoof someone else's attendance.

Q: Is face recognition alone a reliable way to mark attendance, or would you add anything?

A: On its own it's good but not bulletproof — no liveness detection currently means a good enough photo could theoretically fool it. For anything higher-stakes than a classroom (like exam-hall identity verification), I'd add liveness detection, and possibly a human spot-check option for low-confidence matches instead of fully automatic marking in every case.

15 One-Page Cheat Sheet (Skim This Right Before Walking In)

Fact	Value
What it is	Web-based, automatic face-recognition attendance system for classrooms/offices
Face detection model	Google MediaPipe Tasks — BlazeFace (blaze_face_short_range.tflite), returns bounding boxes
Face recognition model	Keras-FaceNet — 512-dimensional embeddings, L2-normalized
Matching method	Cosine similarity (dot product of normalized vectors); default threshold 0.6
Registration flow	15–20 webcam frames → detect + crop face in each → embed each → average into 1 vector → store
Recognition flow	1 frame → detect + crop → embed → compare vs. every stored embedding → mark attendance if above threshold
Why no retraining needed	Recognition is vector comparison, not a trained classifier — unlike the old LBPH approach it replaced
Database	PostgreSQL — 4 tables: users, students, attendance, face_embeddings
Key relationships	attendance & face_embeddings both FK → students.student_id; face_embeddings has ON DELETE CASCADE
Auth	Flask-Login (session cookies) + Werkzeug password hashing; has a hardcoded admin/admin first-run bootstrap (known limitation)
Backend	Flask app factory pattern + Blueprints (auth, students, attendance, face); PostgreSQL via Flask-SQLAlchemy
ML models loaded as	Module-level singletons (get_detector(), get_facenet()) — loaded once per server process
Frontend	WebRTC webcam capture + vanilla JS, Jinja2 server-rendered templates (not a SPA)
Biggest known limitation	Brute-force linear search over all stored embeddings — needs a vector index (pgvector/FAISS) to scale

*Good luck — this project genuinely covers applied computer vision (two-model detection + recognition pipeline), backend engineering (Flask, Blueprints, authentication), and database design (relational integrity, cascading deletes). Lean into explaining **why** the embedding-based approach beats the old classifier-based one, and be ready to name its real limitations yourself — that's what will set you apart.*